

WPSBatch

浏览器插件化方案报告

把当前可跑通的 WPS 表格附件批量改名下载脚本，改造成可复用、可配置、可预览、可诊断的 Chrome/Edge 扩展。

Codex

V1.0 · 2026.05.10

基于 wps-batch-rename-download.js 与当前 KDocs 实测路径

| 目录

01	执行摘要	03
02	现有脚本能力与边界	04
03	插件架构方案	06
04	通用表格适配与执行流程	09
05	风险、回滚与实施路线	11

| 执行摘要

当前脚本已经证明了核心路径可行：可以从 WPS 表格读取任务、匹配附件 ID、更新单元格显示名、修改附件对象名并触发下载。下一步不应把 4493 行脚本简单塞进扩展按钮，而应拆成 页面内核、内容脚本、后台服务、弹窗 UI 四层，让它变成可迁移到其他表格的工具。

核心 Takeaways

- 插件第一版建议只做 Chrome/Edge Chromium，使用 Manifest V3，权限收敛在 `scripting`、`activeTab`、`downloads`、`storage` 与 KDocs 域名。
- “换一个表格也能跑”的关键不是硬编码列号，而是表头别名、命名模板、扫描范围、失败策略都可配置。
- 大批量附件可支持，但必须分批扫描、队列下载、执行前预览；遇到附件 ID 只能靠顺序兜底时，应进入人工确认状态。
- 最大的长期风险来自 WPS 内部 API 变动；插件要把诊断报告产品化，而不是假设接口永远稳定。

CURRENT SCRIPT

4493 行

已包含扫描、匹配、改名、下载和诊断报告。

DEFAULT SCAN

10000 × 60

默认最大行数与列数，可在插件配置中暴露。

DECISION

内部自用版先行

先稳定业务闭环，再考虑发布渠道。

本文档回答的问题

它能不能在别的 KDocs 表格里用？列不在 C/F 时怎么办？很多行很多附件时怎么避免误改、漏下和浏览器拦截？这三件事决定插件是否值得做。

现有脚本能力与边界

这份脚本已经不是一次性下载器，而是一个在控制台中运行的 WPS 自动化内核。插件化要保留它已经验证过的能力，同时把“需要人肉操作控制台”的部分移到扩展界面。

已验证能力

能力	当前入口	插件化后的归属
读取表格并生成计划	<code>buildPlanDirectlyFromPage()</code>	页面内核: 返回标准化 plan
按表头识别列	<code>maxTableRows</code> 、 <code>maxTableCols</code> 与 header 推断	配置模块: 表头别名、扫描范围
读取附件 ID	<code>makeAutoAttachmentMap()</code> 、 <code>link runs</code> 、 <code>hyperlinks</code> 探测	附件解析器: 输出 <code>match confidence</code>
更新单元格显示名	<code>updateSheetAttachmentCellsByAttachmentMap()</code>	WPS mutator: 执行前写入回滚记录
改附件对象名并下载	<code>renameByAttachmentMap()</code> 、 <code>downloadByAttachmentMap()</code>	页面内核改远端名，后台服务负责下载队列
诊断报告	<code>reportTableState()</code> 、 <code>reportHyperLinkMethodSurface()</code> 等	报告模块: 一键复制、导出 JSON

核心问题

脚本能在当前文件跑通，不等于它已经是通用工具。通用化的关键约束有三个：表格结构不固定、附件 ID 不总是直接暴露、WPS 内部 API 没有公开稳定契约。

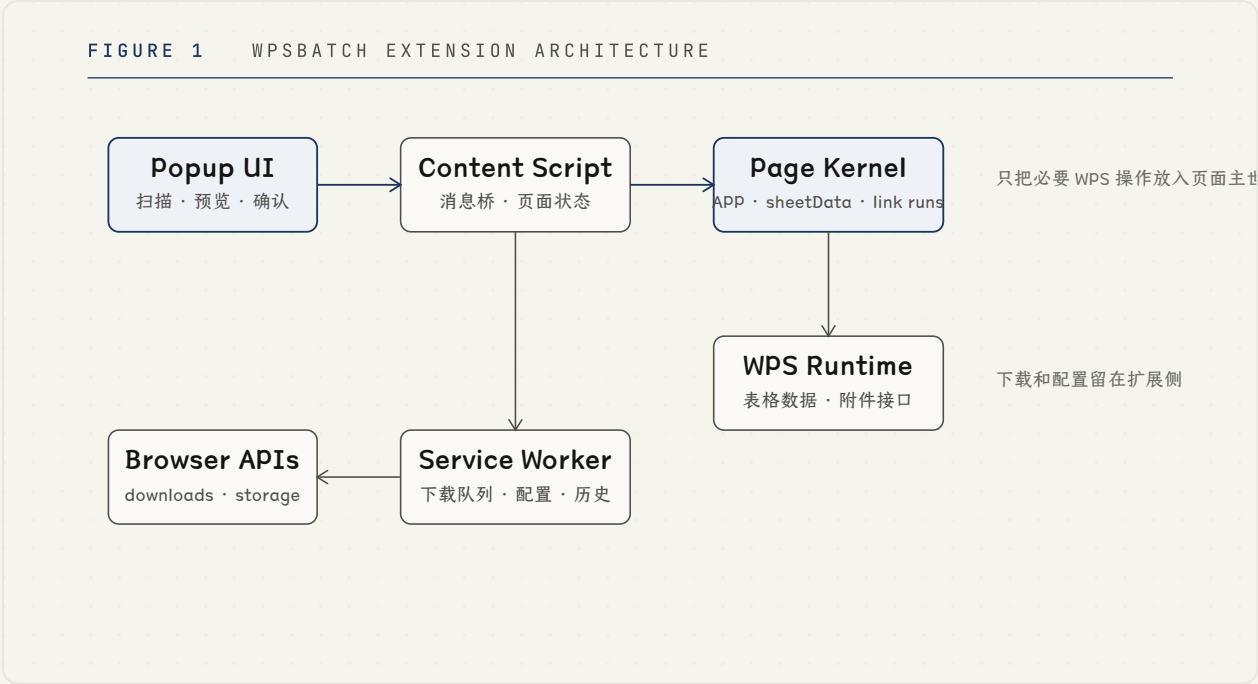
插件可以做到“换文档后仍然大概率可用”，但不应该承诺“任意表格完美运行”。正确目标是：常见模板自动识别；不确定时给预览和异常原因；高风险匹配必须让用户确认。

衡量标准

维度	控制台脚本	插件目标	验收方式
列位置变化	已不完全依赖 C/F，但配置隐藏在脚本里	表头别名可配置，扫描预览显示列命中	移动附件列后仍能生成正确 plan
大批量任务	默认 10000 行、60 列，下载节流 700ms	队列执行、进度条、失败重试	500 行、1000 个附件任务不阻塞 UI
误匹配防护	已有 <code>diagnostics</code> ，但仍偏调试	按置信度分级：确定、需确认、失败	顺序兜底匹配不会无提示执行

插件架构方案

推荐使用 Chrome Manifest V3, 把现有脚本拆成四层: 页面注入脚本访问 WPS 内部对象, content script 做消息桥, service worker 管下载和持久化, popup 负责预览和确认。



插件四层架构: 页面主世界访问 WPS 内部状态, 扩展侧处理下载、配置和用户确认。

Manifest V3 权限建议

```
{
  "manifest_version": 3,
  "permissions": ["scripting", "activeTab", "downloads", "storage"],
  "host_permissions": ["https://www.kdocs.cn/*"],
  "background": { "service_worker": "background.js" },
  "action": { "default_popup": "popup.html" },
  "content_scripts": [{
    "matches": ["https://www.kdocs.cn/*"],
    "js": ["content-bridge.js"],
    "run_at": "document_idle"
  }]
}
```

模块拆分

模块	职责	从现有脚本迁移的内容
core/table-reader	表头识别、读取行、生成 plan	表格 native API 与 DOM fallback
core/name-rules	命名模板、日期/箱数解析、文件名清洗	sanitizeFilename()、target 生成逻辑
core/attachment-resolver	读取 link runs、hyperlink、probe 结果，输出附件映射	makeAutoAttachmentMap() 与 diagnostics
core/wps-mutator	调用 WPS 命令更新显示名和附件对象名	range.setFormula 与附件 name API
background-downloads	下载队列、重名策略、失败监听	替代页面 <a download> 触发下载
reporting	预览、执行结果、异常报告、回滚记录	各类 report* 函数

为什么不能只做 content script

Chrome content script 默认在隔离环境运行，不能天然访问页面 JS 里的 window.APP、sheetData、getTextLinkRuns 等对象。可以使用运行时注入或主世界脚本，但主世界也有被页面影响的风险。因此页面脚本应尽量薄，只暴露 scan、resolve、mutate 这类受控命令。

插件的工程重点不是“能注入代码”，而是把可变的 WPS 内部依赖包起来：一旦接口变动，用户看到的是明确诊断，而不是控制台里一串不可读的异常。

通用表格适配与执行流程

插件可以不固定 C 列和 F 列，但必须把“识别列、识别规则、异常处理”从脚本常量变成用户可理解的配置。它的通用性来自配置和预览，不来自猜测。

换文档后的适配规则

场景	插件行为	能否自动执行
列顺序变化，但表头仍可识别	按表头别名定位列，预览显示命中的列名和列号	可以
表头叫法变化，例如“箱标附件”“预约文件”	用户在配置中加入别名，之后同类表格自动识别	配置后可以
没有表头，或表头合并/跨行	进入手动映射模式：用户指定货件号、时间、箱数、附件列	确认后可以
附件 ID 无法从 link runs 或接口中读取	生成诊断报告，停止改名；禁止靠猜测直接执行	不可以
只能按下载点击顺序推断 ID	标记为高风险，需要用户逐项或批量确认	确认后谨慎执行

用户执行步骤

1. 打开目标 KDocs 表格，等待页面完全加载。
2. 点击扩展图标，插件检测当前页面是否为可编辑 WPS 表格。
3. 点击“扫描表格”，生成任务计划；界面显示行号、货件号、附件类型、源名称、目标名称。
4. 检查列识别结果。如果列名没有命中，在配置里补充表头别名或手动选择列。
5. 点击“匹配附件 ID”，插件读取 link runs、hyperlink 与附件接口，生成 attachment map。
6. 查看异常项。所有 missing-id 或低置信度匹配必须处理完，才能进入执行。
7. 点击“执行”，插件先更新表格显示名，再改 WPS 附件对象名，最后进入下载队列。
8. 完成后导出执行报告。报告包含原值、目标值、附件 ID、下载状态、失败原因和回滚所需信息。

默认配置

命名模板

外箱标： 外箱标-货件号-箱数箱

预约信： 预约信-货件号-时间

扩展名： 默认 .pdf ，如下载接口返回其他扩展名则以接口为准。

扫描与下载

最大扫描： 10000 行、60 列。

空行停止： 连续空行达到阈值后停止。

下载节流： 默认 700ms ，大表可提高。

关键产品判断

“完美运行”的定义不应是插件永远不问用户，而是插件能清楚地区分：哪些可以自动执行，哪些需要补配置，哪些必须停止。这个判断比多写几个 fallback 更重要。

风险、回滚与实施路线

第一版目标是内部自用稳定，而不是立即上架商店。先把控制台脚本变成可重复、可配置、可预览的扩展，再逐步补下载体验和历史记录。

风险与控制

风险	表现	控制方式
WPS 内部 API 变动	<code>getTextLinkRuns</code> 、 <code>range.setFormula</code> 或附件接口失效	保留多路径探测与报告，失败时给出断点位置
附件 ID 误匹配	文件名改到错误附件上	引入置信度；顺序匹配默认需要确认
批量下载被浏览器限制	下载中断、重名、用户被浏览器提示拦截	service worker 队列、节流、失败重试、清晰提示
权限过宽	用户不敢安装或浏览器提示过重	只请求 KDocs 域名与必要权限；优先 activeTab
部分执行失败	表格显示名已改，附件名或下载失败	执行前写回滚记录；结果报告逐项列出状态

回滚记录

所有会改远端数据的动作都要在执行前记录旧值：原单元格文本、原 `kw:annex` 地址、原附件对象名、新目标名、附件 ID、行列位置、执行状态。第一版至少导出 JSON 报告；第二版可以增加“按上次记录恢复名称”。

开发阶段

1. 页面内核化：把当前 `WPSBatch` 拆成 `scan`、`resolve`、`mutate`、`rename`、`download`、`report` 六类 API。
2. 最小扩展：popup 提供扫描、执行、复制报告；content script 完成页面桥接。
3. 下载迁移：用 `chrome.downloads` 接管下载队列、文件名、失败监听和重名策略。
4. 配置与历史：加入表头别名、命名模板、扫描上限、下载目录、执行历史。
5. 内部分发：先以解压扩展或企业策略安装使用，暂不优先上 Chrome Web Store。

最低测试集

- 当前已跑通的测试表。
- 附件列不在 C/F 的表格。
- 至少 500 行、1000 个附件任务的大表。
- 缺失附件、只读页面、禁止下载页面。
- 表头别名变化，例如“箱标附件”“预约文件”。
- 名称含多余文字，例如“测试的”。
- WPS 页面刚打开未完全加载时点击扩展。
- 执行中刷新、断网或单个附件接口失败。

A. 参考资料

- Chrome 官方文档: `chrome.scripting` API, 用于运行时注入脚本, 需声明 `scripting` 权限并获得目标页面权限。
- Chrome 官方文档: `Manifest content scripts`, 说明 `content script` 的匹配、注入时机与执行环境。
- Chrome 官方文档: `Message passing`, 说明扩展各脚本之间的消息通信方式。
- Chrome 官方文档: `chrome.downloads` API, 用于程序化发起、监控和管理下载。

B. 术语表

Page Kernel: 注入到 WPS 页面主世界的业务内核, 负责访问 `window.APP` 和 WPS 内部数据。

Attachment Map: 以附件类型和货件号为键, 记录附件 ID、目标名称、原始名称和诊断状态的映射。

Link Runs: WPS 单元格富文本链接信息, 通常包含附件的 `kw:annex` 地址或可解析的子文件 ID。

低置信度匹配: 无法通过单元格链接直接确认, 只能通过顺序或历史点击推断的附件匹配结果。

C. 结论

建议立即进入第一阶段: 先把脚本拆成页面内核 API, 并保留现有控制台入口作为回归测试工具。只要扫描、匹配、更新、改名、下载都能通过同一套结构化结果返回, 插件外壳就会变得很自然; 反过来, 如果先做 UI, 再把 4493 行脚本整体塞进去, 后续每一次 WPS 接口变化都会变成维护成本。